Федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
Физико-механический институт
Высшая школа прикладной математики и вычислительной физики

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

по курсу «Введение в технологии суперкомпьютерных вычислений»
на тему: «Параллелизация алгоритмов выполнения типовых операций с одномерными массивами»

Работу выполнила:
Студент гр. 5030301/40102
Набережных Виктория Александровна

Преподаватели:
доцент ВШПМиВФ, к.т.н. Абрамов Алексей Геннадьевич
студент магистратуры ВШПМиВФ Мовисян Максим Артурович

Санкт-Петербург
2026

Оглавление

1. Цель работы и постановка задачи 3

2. Информация об аппаратном и программном обеспечении 3

3. Исследование оптимизации компилятора и автоматического распараллеливания 4

5. Исходные коды разработанных программ с комментариями к распараллеленным областям и используемым конструкциям OpenMP 6

6. Результаты проверки правильности выполнения программы 8

7. Результаты исследования эффективности распараллеливания программы 8

8. Результаты исследования эффективности способов распределения итераций цикла 10

Выводы 10

Приложение (код программы) 11

1. Цель работы и постановка задачи

Целью данной работы является освоение средств и методики распараллеливания алгоритмически относительно простых задач с помощью технологии OpenMP на примере выполнения типовых операций с одномерными массивами.

Для достижения поставленной цели необходимо разработать программу, реализующую типовую операцию с одномерными массивами (векторами) и выполнить ее распараллеливание средствами OpenMP.

Математическая постановка задачи: вычислить сумму S всех элементов одномерного массива (вектора) вещественных чисел двойной точности размером N:

$$S = \sum_{i=1}^{N} v_i$$

где v_i – элемент исходного вектора элементов.

В программе, написанной на языке Fortran, индексация элементов массива ведется от 1 до N. Чтобы операции с памятью и вычисления занимали заметное процессорное время (порядка 0.1–1.0 секунды) и накладные расходы на создание потоков не перекрывали полезную работу, размер массива выбирается достаточно большим: N = 10^8 элементов. Массив инициализируется тестовыми данными по формуле:

$$v_i = \frac{1{,}0}{i}$$

Для анализа производительности введены следующие метрики:

1. *Время выполнения Tp* – измеряется с помощью функции реального(астрономического) времени omp_get_wtime(), которая возвращает время в секундах (тип real(8))

2. *Ускорение Sp* - на p потоках по отношению к последовательной версии T1:

$$Sp = \frac{T1}{Tp}$$

3. *Эффективность распараллеливания Ep:*

$$Ep = \frac{Sp}{p} = \frac{T1}{p \cdot Tp}$$

В идеальном случае $Sp \approx p$, а $Ep \approx 1.0$ (100%).

2. Информация об аппаратном и программном обеспечении

Для проведения вычислительных экспериментов и замера времени работы программы использовался ноутбук на базе операционной системы macOS. Написание и редактирование исходного кода выполнялось в текстовом редакторе Geany, а автоматизация прогонов - с помощью командных скриптов Bash.

Аппаратная конфигурация компьютера:

Процессор: Apple M1 (архитектура ARM64, 8 ядер: 4 производительных и 4 энергоэффективных)

Оперативная память: 16 ГБ единой памяти (Unified Memory)
Максимальное число потоков: в среде OpenMP процессор определяется как 8-поточный

Использованное программное обеспечение:

Компилятор: gfortran (GNU Fortran, установлен через пакетный менеджер Homebrew)

Ключи компиляции (Release): флаг -O3 для включения максимальной оптимизации кода компилятором и флаг -fopenmp для поддержки многопоточных директив

Инструменты анализа правок: среда Jupyter Notebook, язык Python 3 с библиотеками pandas (обработка таблиц) и matplotlib (визуализация и графики)

3. Исследование оптимизации компилятора и автоматического распараллеливания

Было исследовано, как компилятор gfortran умеет самостоятельно оптимизировать код работы с массивами и может ли он автоматически распределить цикл по ядрам без явного использования многопоточных директив OpenMP.

Эксперимент проводился на последовательном алгоритме (SERIAL) при фиксированном числе разбиений N = 100 000 000. Программа была скомпилирована и запущена в двух режимах:

1. *Режим Debug* (без оптимизации, флаг -O0):

```
gfortran -O0 -fopenmp lab2.f90 -o vector_debug
```

Компиляция выполняется в соответствии с последовательностью исходного кода. Время выполнения составило 0,406788 сек.

2. *Режим Release* (с максимальной оптимизацией, флаг -O3):

```
gfortran -O3 -fopenmp lab2.f90 -o vector_program
```

Компилятор выполняет глубокий анализ кода, разворачивает мелкие циклы и задействует специальные векторные SIMD-инструкции процессора для одновременной обработки нескольких чисел за один такт. Время выполнения составило 0.110952 сек.

Таким образом, можно сказать, что включение флага -O3 сократило время последовательного расчета с 0,406 до 0,110 секунд — то есть код ускорился в 3,67 раза (на 72,7%). Такой мощный прирост скорости объясняется тем, что компилятор с флагом -O3 смог применить глубокую конвейерную оптимизацию. Вместо поштучного сложения каждого элемента массива, процессор M1 задействовал аппаратные векторные регистры и стал складывать по несколько вещественных чисел двойной точности одновременно за

одну инструкцию, что кардинально снизило накладные расходы на переходы по индексам цикла.

Попытка заставить компилятор автоматически распараллелить последовательный код (без написания директив OpenMP) не успешна. За счет флага -O3 время последовательного метода упало до 0,110 секунды, однако этот прирост произошел исключительно за счет локальной векторной математики внутри одного ядра. Компилятор не задействовал другие ядра процессора, так как время параллельного метода REDUCTION на 4 потоках составило всего 0,031 секунды. Если бы компилятор сам автоматически включил многопоточность для SERIAL, был бы получен такой же быстрый результат. Раз этого не произошло, значит, весь цикл считался строго на одном ядре. Это доказывает необходимость явного использования технологии OpenMP.

4. Формулировка параллельного алгоритма для соответствующей операции

В данной работе выполняется базовая операция над векторами — вычисление суммы всех элементов одномерного массива (вектора) вещественных чисел двойной точности. Размер вектора составляет $N = 10^8$ элементов.

Параллельный алгоритм раскидывает индексы массива между запущенными потоками OpenMP и работает в несколько этапов:

Распределение витков цикла (итераций):

Весь массив от 1 до N нужно поделить между потоками. В работе реализовано два подхода:

В ручном методе (SPMD) делим индексы циклически с шагом, равным числу потоков. Поток с номером tid берет элементы «через один»: tid + 1, затем tid + 1 + p, tid + 1 + 2p и так далее, пока не дойдет до конца массива.

В автоматическом методе (REDUCTION) не считаем индексы вручную — директива !$omp parallel do заставляет планировщик OpenMP самостоятельно нарезать массив на непрерывные блоки и раздать их ядрам.

Локальные вычисления:

Каждый поток работает полностью независимо на своем куске массива. Потоки читают значения элементов из общей памяти (vector(i)) и копят свою промежуточную сумму в личную (приватную) переменную. Так как у каждого потока своя переменная, они не мешают друг другу в процессе расчетов.

Синхронизация результатов:

Когда цикл заканчивается, у каждого потока на руках остается своя частичная сумма. Их нужно сложить вместе в одну общую переменную. Чтобы потоки не полезли туда одновременно и не сломали данные (избегаем Data Race), используются два механизма:

Конструкция atomic (в методе SPMD): сложение локальных сумм в общий итог защищается директивой !$omp atomic. Это быстрая неделимая инструкция, которая выполняется прямо на уровне железа (процессора).

Конструкция reduction (в методе REDUCTION): опция reduction(+:s) создает скрытые копии переменной суммы для каждого потока, а на выходе из цикла сама безопасно и максимально быстро складывает их в итоговую переменную.

Финальный результат:

После слияния всех локальных сумм в главной переменной оказывается точная сумма всех 100 миллионов элементов массива. Главный поток завершает работу и выводит на экран итоговое число и время, которое ушло на расчеты.

№	Вариант	Суть подхода к реализации операции
1	Последовательный (SERIAL)	Обычный цикл. Сумма считается строго на одном ядре процессора, OpenMP не используется.
2	SPMD-вариант (SPMD)	Ручной метод. Потоки запускаются через !$omp parallel, сами узнают свой номер tid, делят массив сеткой и в конце складывают результаты через !$omp atomic.
3	Цикл с редукцией (REDUCTION)	Автоматический метод. Цикл параллелится директивой !$omp parallel do, а безопасный сбор итоговой суммы берет на себя компилятор через опцию reduction(+:s).

5. Исходные коды разработанных программ с комментариями к распараллеленным областям и используемым конструкциям OpenMP

Полный код программы представлен в приложении. Рассмотрим коды расчетных функций на языке Fortran, реализующие последовательный, SPMD и REDUCTION варианты.

Во всех параллельных версиях соблюдены следующие принципы:

Конструкции private используются для полной изоляции внутренних переменных внутри каждого потока (индексы циклов, локальные накопители). Это нужно, чтобы потоки не затирали данные друг друга.

Конструкции shared дают всем потокам общий доступ к неизменяемым параметрам - самому исходному массиву элементов (vec) и его размеру (n).

1. Последовательный алгоритм (SERIAL)

Производится один обычный цикл do от 1 до N. Программа выполняется строго последовательно на одном ядре процессора, директивы OpenMP тут не вызываются.

2. SPMD-вариант

В этой функции параллельная область запускается вручную с помощью директивы !$omp parallel. Внутри нее каждая нить через специальные функции узнает свой личный номер (tid) и общее количество созданных потоков (nt). Локальный накопитель local_sum у каждого потока свой. Элементы массива распределяются сеткой: потоки идут по вектору со сдвигом на nt. Когда цикл заканчивается, потоки по очереди скидывают свои локальные

суммы в общий итог. Чтобы они не сделали это одновременно, поставлена директива !$omp atomic — она выполняет сложение на уровне одной быстрой инструкции процессора и защищает от гонки данных.

3. Автоматический параллельный цикл с редукцией (REDUCTION)

В этом варианте работу по нарезке индексов берет на себя компилятор. Директива !$omp parallel do автоматически делит 100 миллионов элементов массива на равные блоки и раздает их потокам. Для сборки финального ответа использовалась опция reduction(+:s), которая сама создает скрытую локальную копию переменной s для каждого потока, а при выходе из цикла безопасно и эффективно складывает их вместе. Это избавило от необходимости вручную писать вызовы.

```
!последовательный (Sequential) обычный цикл на одном потоке без директив OpenMP.
function sum_sequential(vec, n) result(res)
    implicit none
    integer(8), intent(in) :: n
    real(8), intent(in) :: vec(n)
    real(8) :: res, s
    integer(8) :: i

    s = 0.0d0
    do i = 1, n
        s = s + vec(i)
    end do
    res = s
end function sum_sequential
```

1.Последовательный (SERIAL)

```
!SPMD (ручное циклическое распределение)
function sum_spmd(vec, n) result(res)
    use omp_lib
    implicit none
    integer(8), intent(in) :: n
    real(8), intent(in) :: vec(n)
    real(8) :: res, total_sum, local_sum
    integer(8) :: i
    integer :: tid, nt

    total_sum = 0.0d0
    !$omp parallel private(tid, nt, i, local_sum) shared(vec, n, total_sum)
    tid = omp_get_thread_num()
    nt = omp_get_num_threads()
    local_sum = 0.0d0

    ! распределение витков цикла вручную с шагом nt
    do i = tid + 1, n, nt
        local_sum = local_sum + vec(i)
    end do

    ! безопасное (атомик) сложение локальных результатов нитей в общую переменную
    !$omp atomic
    total_sum = total_sum + local_sum
    !$omp end parallel

    res = total_sum
end function sum_spmd
```

2.SPMD-вариант

```
!Reduction (вариант с директивой !$omp do (автоматическое распределение))
function sum_reduction(vec, n) result(res)
    use omp_lib
    implicit none
    integer(8), intent(in) :: n
    real(8), intent(in) :: vec(n)
    real(8) :: res, s
    integer(8) :: i

    s = 0.0d0
    ! автоматическая нарезка цикла do между запущенными нитями
    ! reduction(+:s) изолирует промежуточные переменные и суммирует их в конце
    !$omp parallel do reduction(+:s)
    do i = 1, n
        s = s + vec(i)
    end do
    !$omp end parallel do

    res = s
end function sum_reduction
```

3.Parallel Do + Reduction (REDUCTION)

6. Результаты проверки правильности выполнения программы

Чтобы убедиться, что параллельный код работает правильно и нигде не теряет числа, я сравнил итоговые суммы элементов массива во всех режимах запуска. Тестовая формула для инициализации элементов вектора $\frac{1,0}{i}$.

Одиночный последовательный метод (SERIAL), ручной SPMD-вариант и автоматический REDUCTION при размере массива в 100 000 000 элементов выдали в консоль одинаковое число:

$$S_{расчет} = 18{,}997896414$$

Тот факт, что все три версии выдали совпадение до последнего знака, доказывает, что в параллельных функциях полностью отсутствует гонка данных. Элементы вектора не затирались и не терялись в процессе вычислений. Это значит, что и ручные атомарные операции !$omp atomic, и встроенный механизм reduction работают математически корректно, а потоки правильно делят между собой оперативную память.

7. Результаты исследования эффективности распараллеливания программы

С помощью языка программирования Python и библиотек Pandas, Matplotlib были построены графики ускорения и эффективности для всех вариантов от числа потоков (см. Рис.1). Результаты работы каждого варианта представлены в таблице 1.

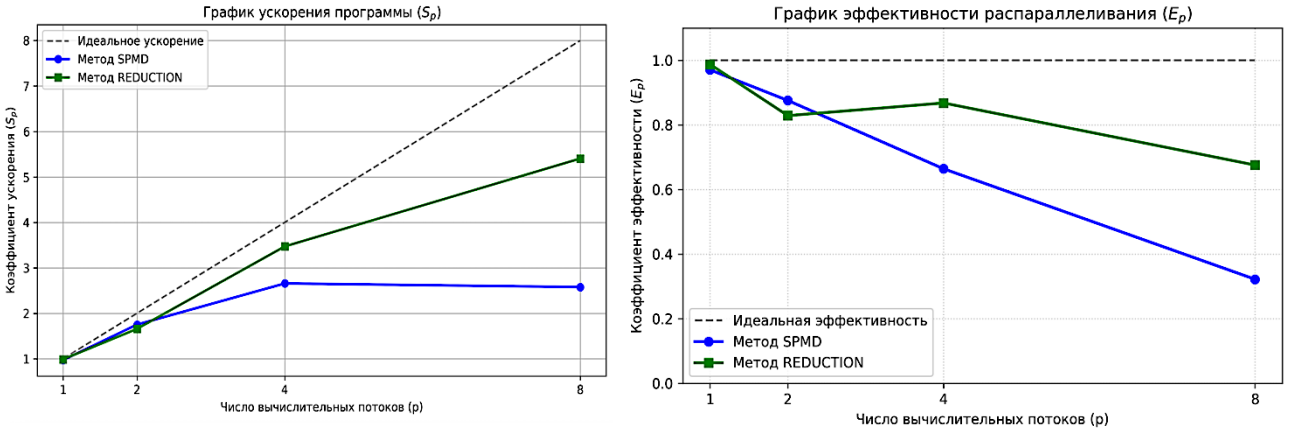


Рисунок 1. Графики ускорения и эффективности от числа потоков

На 2 потоках время упало почти в два раза (до 0,063–0,066 сек). Ускорение составило 1,66–1,75. Нагрузка распределяется по мощным ядрам равномерно, а накладные расходы на синхронизацию минимальны.

На 4 потоках метод REDUCTION ушел далеко вперед ручного SPMD, показав время 0,031 сек и ускорение 3,47. Эффективность REDUCTION подскочила до 87%, так как блоки данных по 25 млн элементов начали идеально укладываться в быстрый кэш процессора. В SPMD из-за циклического шага нитей начались промахи мимо кэша, и ядра простаивали.

На 8 потоках REDUCTION выдал максимум скорости (0,020 сек), а его ускорение составило 5,40. Метод SPMD на 8 потоках затормозился (0,043 сек), а его эффективность снизилась до 32%.

Отсутствие линейного ускорения в 8 раз - норма для процессора Apple M1. Его вторая половина ядер (4 ядра Efficiency) изначально сделана более слабой и низкочастотной, поэтому они не могут дать сильного прироста, а накладные расходы на сборку суммы возрастают.

Метод параллелизации	Число потоков p	Время выполнения (сек)	Ускорение Sp	Эффективность Ep = Sp/p
SERIAL	1	0.110952000	1.00	1.00 (100%)
SPMD	1	0.114370000	0.97	0.97 (97%)
SPMD	2	0.063379000	1.75	0.88 (88%)
SPMD	4	0.041779000	2.66	0.67 (67%)
SPMD	8	0.043052000	2.58	0.32 (32%)
REDUCTION	1	0.112385000	0.99	0.99 (99%)
REDUCTION	2	0.066967000	1.66	0.83 (83%)
REDUCTION	4	0.031972000	3.47	0.87 (87%)
REDUCTION	8	0.020532000	5.40	0.68 (68%)

Таблица 1. Результаты работы всех 3 вариантов

8. Результаты исследования эффективности способов распределения итераций цикла

Была проведена проверка, как размер блока распределяемых итераций (опция schedule) при работе 2 потоков влияет на скорость метода REDUCTION на 100 млн элементов массива (см. рис.2):

```
OMP_NUM_THREADS=2 OMP_SCHEDULE="static,1" ./vector_program REDUCTION 100000000
OMP_NUM_THREADS=2 OMP_SCHEDULE="static,10" ./vector_program REDUCTION 100000000
OMP_NUM_THREADS=2 OMP_SCHEDULE="static,1000" ./vector_program REDUCTION 100000000
```

При размере блока 1 (static,1) время составило 0,056093 сек. Оно чуть выше, так как нарезка цикла на порции по 1 элементу заставляет процессор часто переключаться.

При размере блока 10 (static,10) время осталось примерно таким же 0,056828 сек.

При укрупнении блоков до 1000 (static,1000) время упало до минимума 0,054133 сек. Это лучший результат, потому что нагрузка на всем массиве одинаковая и ядра загружены равномерно.

Для простых операций над массивами, где сложность расчета одинакова для любого элемента, лучшим выбором является крупноблочная статика (static,1000). Динамическое распределение здесь не нужно, так как оно тратит ресурсы процессора на лишнее управление очередью задач.

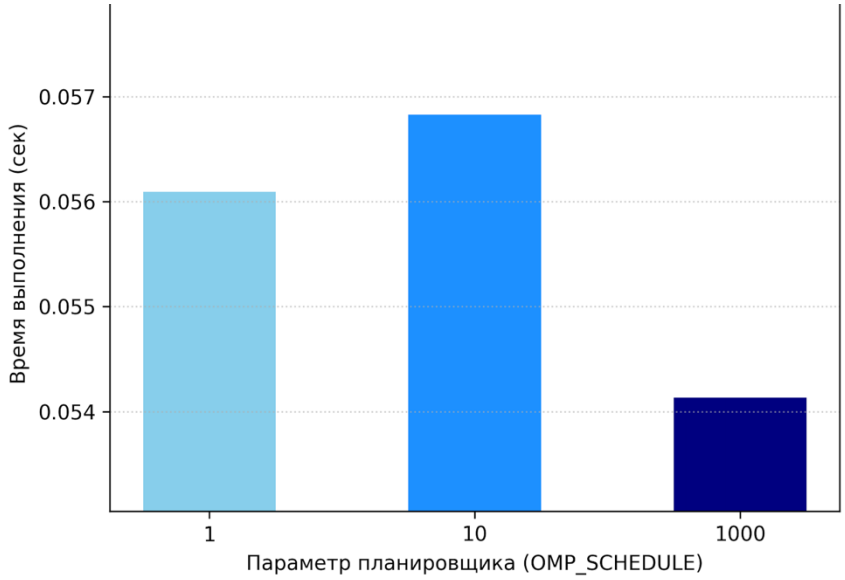


Рисунок 2. Влияние размера блока статического планировщика на время работы

Выводы

В ходе лабораторной работы были изучены базовые конструкции OpenMP при распараллеливании алгоритмов выполнения типовых операций с одномерными массивами для варианта №4. Включение оптимизации компилятора с флагом -O3 сократило время последовательного расчета с 0.406 до 0.110 секунды за счет автовекторизации (SIMD) цикла обработки элементов вектора. Автоматическое распараллеливание силами одного лишь компилятора не сработало из-за циклической зависимости по данным в переменной итоговой суммы, что подтвердило необходимость ручного применения OpenMP.

Все разработанные параллельные модификации показали высокую скорость работы и выдали одинаковый математический результат суммы (18.997896414), что доказывает отсутствие гонки данных и правильность расчетов. Автоматический метод REDUCTION показал себя значительно эффективнее ручного SPMD. На 2 и 4 потоках для REDUCTION было получено хорошее ускорение (в 1.66 и 3.47 раза), при этом на 4 потоках эффективность подскочила до 87%, так как блоки данных оптимально уложились в разделяемый кэш быстрых ядер. На максимальных 8 потоках рост производительности у REDUCTION замедлился до 5.40 раз, а у SPMD вообще затормозился на 2.58, что связано с разнородной архитектурой процессора M1, у которого только половина ядер являются высокопроизводительными, а остальные четыре — энергоэффективными. В ручном SPMD из-за циклического шага нитей по массиву происходили постоянные промахи мимо кэша, что привело к падению его эффективности до 32%.

Исследование размеров блока статического планировщика на 2 потоках показало, что для простых циклов с равномерной нагрузкой лучше всего подходит крупноблочное статическое распределение итераций с параметром static,1000, которое выдало минимальное время 0.054133 секунды. Режимы static,1 и static,10 отработали медленнее (0.056093 и 0.056828 секунды), так как при мелкой нарезке вектора процессор тратил лишнее время на постоянное управление очередью и выдачу мелких порций задач потокам прямо во время вычислений.

Приложение (код программы)

```
module variants_lib
   implicit none

   integer, parameter :: vUNKNOWN = 0 !метод для отслеживания ошибок
   integer, parameter :: vSERIAL = 1 !последовательный алгоритм
   integer, parameter :: vSPMD = 2 !SPMD
   integer, parameter :: vREDUCTION = 3 !автоматический цикл с редукцией

   contains

   !превращает константы vSERIAL, vSPMD,... в текстовое слово
   function var2str(var) result(str)
      integer, intent(in) :: var
      character(len=:), allocatable :: str
      select case (var)
      case (vSERIAL); str = "SERIAL"
      case (vSPMD); str = "SPMD"
      case (vREDUCTION); str = "REDUCTION"
      case default; str = "UNKNOWN"
      end select
   end function var2str
  !аналогично в обратную сторону
   function str2var(str) result(var)
      character(len=*), intent(in) :: str
      integer :: var
      select case (str)
      case ("SERIAL"); var = vSERIAL
      case ("SPMD"); var = vSPMD
```

```
        case ("REDUCTION"); var = vREDUCTION
        case default; var = vUNKNOWN
        end select
    end function str2var

end module variants_lib

!начало основной программы
program main
    use variants_lib
    use omp_lib, only: omp_get_wtime, omp_get_max_threads
    implicit none

    integer, parameter :: Action_RUN = 0, Action_ERROR = 1 !флаги проверки правильности
программы

    type :: Options !структура настройки, внутри код выбранного метода (variant) и число
разбиений интеграла (ndiv)
        integer :: variant = vUNKNOWN
        integer(8) :: ndiv = 0 !размер массива N
    end type Options

    type ParseResult !структура результат разбора: флаг действия (action), настройки
(options),текстовые поля под описание ошибки (error) и аргумент, который эту ошибку
вызвал (argument)
        integer :: action = Action_RUN
        type(Options) :: options
        character(len=:), allocatable :: error
        character(len=:), allocatable :: argument
    end type ParseResult

    type(ParseResult) :: parsed !итог проверки аргументов терминала
    character(len=256) :: prog !строка для хранения имени запущенной программы
    character(len=32) :: Sum_str, Time_str
    integer(8) :: n, i
    real(8) :: total_sum, exec_time

    ! динамический массив (вектор)
    real(8), allocatable :: vector(:)

   !запуск всех вариантов вычислений
    interface
        function sum_sequential(vec, n) result(res)
            integer(8), intent(in) :: n
            real(8), intent(in) :: vec(n)
            real(8) :: res
        end function
        function sum_spmd(vec, n) result(res)
            integer(8), intent(in) :: n
            real(8), intent(in) :: vec(n)
            real(8) :: res
        end function
```

```
        function sum_reduction(vec, n) result(res)
            integer(8), intent(in) :: n
            real(8), intent(in) :: vec(n)
            real(8) :: res
        end function
    end interface

!вывод в консоль ошибок
    call get_command_argument(0, prog)
    parsed = parse_input_arguments()

    if (parsed%action == Action_ERROR) then
        call report_error(trim(prog), parsed%error, parsed%argument)
        stop 1
    end if

    n = parsed%options%ndiv
    total_sum = 0.d0

    allocate(vector(1:n))

   !заполнение массива тестовыми данными (вынесено из замера времени)
    !$omp parallel do
    do i = 1, n
        vector(i) = 1.0d0 / real(i, 8)
    end do

    exec_time = omp_get_wtime() !запуск секундомера
    select case (parsed%options%variant) !переключатель кода метода
    case (vSERIAL); total_sum = sum_sequential(vector, n)
    case (vSPMD); total_sum = sum_spmd(vector, n)
    case (vREDUCTION); total_sum = sum_reduction(vector, n)
    case default; continue
    end select
    exec_time = omp_get_wtime() - exec_time !остановка секундомера

!вывод результатов
    write(Sum_str, '(F20.9)') total_sum
    write(Time_str, '(F20.9)') exec_time
    write (*, '(A,",",I0,",",I0,",",A,",",A)') &
        var2str(parsed%options%variant), omp_get_max_threads(),&
        n, trim(adjustl(Sum_str)), trim(adjustl(Time_str))

    deallocate(vector)

    contains
   !защитный механизм-превращает аргумент терминала в число
    subroutine parse_positive_int(token, val, ok)
        character(len=*), intent(in) :: token
        integer(8), intent(inout) :: val
        logical, intent(inout) :: ok
        integer :: i, digit
```

```
        ok = .false.
        val = int(0, 8)
        if (len(token) == 0) return
        do i = 1, len_trim(token)
           if (token(i:i) < '0' .or. token(i:i) > '9') return;
           digit = iachar(token(i:i)) - iachar('0')

           val = val * int(10, 8) + int(digit, 8) !сбор из цифр числа
        end do
        ok = (val > int(0, 8))
    end subroutine parse_positive_int

!запись информации об ошибке в структуру res (флаг Action_Error, сообщение message и
проблемное слово argument)
    subroutine set_failure(res, message, argument)
      type(ParseResult), intent(inout) :: res
      character(len=*), intent(in)     :: message
      character(len=*), intent(in)     :: argument

      res%action = Action_Error
      res%error = message
      res%argument = argument
    end subroutine set_failure

    ! разбор аргументов командной строки и возвращает структуру с результатом парсинга
    function parse_input_arguments() result(result_)
      type(ParseResult) :: result_
      integer :: i, argc, positional_count, arg_len
      character(len=:), allocatable :: token
      logical :: ok

    ! инициализация структуры: по умолчанию ошибок нет
      result_%action = Action_Run
      result_%error = ""
      result_%argument = ""

    ! получение общего количества переданных аргументов
      argc = command_argument_count()
      positional_count = 0

    ! перебор всех аргументов командной строки в цикле
      do i = 1, argc
       ! динамическое выделение памяти под размер текущего аргумента
         if (allocated(token)) deallocate(token)
         call get_command_argument(i, length=arg_len)
         allocate(character(len=arg_len) :: token)
         call get_command_argument(i, value=token)

       ! первый аргумент - должен быть именем метода
         if (positional_count == 0) then
            positional_count = positional_count + 1
```

```
            result_%options%variant = str2var(token)
            if (result_%options%variant == vUNKNOWN) then
               call set_failure(result_, "Invalid directive name", token)
               return
            end if
            cycle

        ! второй аргумент - должен быть числом разбиений
          else if (positional_count == 1 .and. len(token) > 0) then
            positional_count = positional_count + 1
            call parse_positive_int(token, result_%options%ndiv, ok)
            if (.not. ok) then
              call set_failure(result_, "Invalid array size", token)
              return
            end if
            cycle
          end if

        ! все последующие аргументы - ошибка
          call set_failure(result_, "Unexpacted argument", token)
          return
        end do

    ! если введено меньше двух обязательных аргументов- ошибка
      if (positional_count < 2) then
         call set_failure(result_, "Missing required arguments. Expected: NAME N", "")
      end if
    end function parse_input_arguments


    ! выводит сообщение об ошибке, правила использования программы и список
доступных директив
    subroutine report_error(prog_name, message, argument)
      character(len=*), intent(in) :: prog_name
      character(len=*), intent(in) :: message
      character(len=*), intent(in) :: argument
      write (*, '(2A)', advance='no') "Error: ", message
      if (len(argument) > 0) then
         write (*, '(3A)', advance='no') " -> '", argument, "'"
      end if
      write (*, '(A)') new_line('a')
      write (*, '(A)') "Usage: " // prog_name // " <DIRECTIVE NAME> <ARRAY SIZE>"
      write (*, '(A)') "Available names:"
      write (*, '(A)') "  SERIAL SPMD REDUCTION"
    end subroutine report_error

end program main

!последовательный (Sequential) обычный цикл на одном потоке без директив OpenMP.
function sum_sequential(vec, n) result(res)
    implicit none
    integer(8), intent(in) :: n
```

```
    real(8), intent(in) :: vec(n)
    real(8) :: res, s
    integer(8) :: i

    s = 0.0d0
    do i = 1, n
        s = s + vec(i)
    end do
    res = s
end function sum_sequential

!SPMD (ручное циклическое распределение)
function sum_spmd(vec, n) result(res)
    use omp_lib
    implicit none
    integer(8), intent(in) :: n
    real(8), intent(in) :: vec(n)
    real(8) :: res, total_sum, local_sum
    integer(8) :: i
    integer :: tid, nt

    total_sum = 0.0d0
    !$omp parallel private(tid, nt, i, local_sum) shared(vec, n, total_sum)
    tid = omp_get_thread_num()
    nt = omp_get_num_threads()
    local_sum = 0.0d0

    ! распределение витков цикла вручную с шагом nt
    do i = tid + 1, n, nt
        local_sum = local_sum + vec(i)
    end do

    ! безопасное (атомик) сложение локальных результатов нитей в общую переменную
    !$omp atomic
    total_sum = total_sum + local_sum
    !$omp end parallel

    res = total_sum
end function sum_spmd

!Reduction (вариант с директивой !$omp do (автоматическое распределение))
function sum_reduction(vec, n) result(res)
    use omp_lib
    implicit none
    integer(8), intent(in) :: n
    real(8), intent(in) :: vec(n)
    real(8) :: res, s
    integer(8) :: i

    s = 0.0d0
    ! автоматическая нарезка цикла do между запущенными нитями
    ! reduction(+:s) изолирует промежуточные переменные и суммирует их в конце
```

```
    !$omp parallel do reduction(+:s)
    do i = 1, n
       s = s + vec(i)
    end do
    !$omp end parallel do

    res = s
end function sum_reduction
```